

SYSTEM CORRUPT / SYCO23

TRANSMISSION SYSTEM · BROADCAST CONTROL · V4.1

SYCO23

MULTICAST CONTROL

FINAL PROMPT PACK

MASTER SPEC

DESIGN PROMPT

AGENT BUILD PROMPT

USER STORIES

ROADMAP

REFERENCE PACK

PROJECTS COVERED

— syco23-webplayer —

— syco23-transmission —

— azuracast —

— syco23bot + adminbot —

SYCO23 V4.1 — FINAL PROMPT PACK

This pack includes the full v4.1 master spec, design prompt, build prompt, reference docs, user stories, and roadmap.

SYCO23 V4.1 — MASTER PRODUCT SPEC

1. System definition

SYCO23 v4.1 is now defined as a multi-project ecosystem rather than one single app. The implementation is split into four connected projects:

1. syco23-webplayer — public listening surface and future public visual preview surface.
2. syco23-transmission — multistream operator webapp and transmission control system.
3. azuracast — station backend installed on the Linode VPS.
4. syco23bot + syco23adminbot — Telegram bot project for public listeners and internal operators.

All projects must follow the same brand system: mechanical totem, ancient visual energy, sound-system scale, dark mineral surfaces, one dominant key color per asset, industrial display typography, functional readable body text, and anti-commercial restraint.

2. Shared v4.1 rules

- No terminal or console metaphors.
 - No neon rave, cyberpunk, glossy startup, or generic SaaS dashboard aesthetics.
 - Mobile-first and fully responsive wherever a visual interface exists.
 - Shared API contracts between projects.
 - Shared brand tone across UI, kits, logs, and bot messages.
 - Clear separation of project boundaries.
-

3. Project separation

3.1 syco23-webplayer

Public-facing listening surface.

Responsibilities

- Live audio playback.
- Metadata presentation.
- Schedule and archive surfaces.
- Offline, reconnecting, and fallback states.
- Future-ready video preview embedding path.
- SYCO23 v4.1 public identity surface.

3.2 syco23-transmission

Operator-facing multistream webapp.

Responsibilities

- Create streams from templates with generated previews.
- Custom template builder with user background assets.
- Custom output profiles.
- Provider management and multistream routing.

- Provider-specific transmission kits.
- Built stream preview player.
- Colorized logs.
- SQLite persistence.
- FFmpeg pipeline management.
- Ultra-efficient watchdog service.
- Metadata middleware for AzuraCast enrichment.

3.3 azuracast

Core radio backend on Linode VPS.

Responsibilities

- Station backend.
- Stream source and relay foundation.
- Public API baseline for metadata.
- Stable service base for downstream tooling.

3.4 syco23bot + syco23adminbot

Telegram bot layer.

Responsibilities

- Public now-playing and link distribution.
- Admin alerts from watchdog and transmission events.
- Admin status queries.
- Controlled remote actions.
- Bot audit trail.

4. Core features by project

syco23-webplayer

- Live playback.
- Mobile-first responsive UI.
- Metadata stack.
- Schedule and archive.
- Fallback/offline states.
- Future HLS/video preview integration.

syco23-transmission

- Template gallery with generated previews.
- User template builder.
- Provider-specific output profiles.
- Multistream routing to YouTube, Telegram, TikTok, Twitch, Instagram, Mixer, Mixcloud, Facebook, and future custom RTMP targets.
- Transmission kit generator.
- Frontend video preview player.
- Colorized log system.
- SQLite persistence.
- FFmpeg pipeline orchestration.

- Watchdog supervision.
- AzuraCast metadata middleware.

azuracast

- Linode deployment.
- Baseline station and stream source.
- Public API source for middleware.
- Recoverable infrastructure and backup path.

syco23bot + syco23adminbot

- Public listener bot.
 - Internal admin bot.
 - Alerting.
 - Status queries.
 - Safe remote actions.
 - Audit logging.
-

5. Deployment boundaries

- azuracast runs on the Linode VPS.
 - syco23-transmission runs on Ubuntu/Node or Docker; Vercel may host UI-only or stateless surfaces, but not FFmpeg or watchdog long-running services.
 - syco23-webplayer may run on Vercel or similar web hosting depending on integration path.
 - bots run as long-running Node services with secured credentials.
-

6. Persistence and data flow

- AzuraCast provides baseline station/public API data.
 - Transmission middleware normalizes and enriches AzuraCast data into SQLite.
 - Webplayer consumes normalized APIs rather than raw backend internals.
 - Bots consume normalized internal APIs and event streams.
 - Critical operational and creative entities persist in SQLite where applicable.
-

7. Required companion document

Use the separate SYCO23 v4.1 User Stories document as the implementation-ready story backlog for all four projects.

SYCO23 V4.1 — DESIGN LAYOUT PROMPT

Design the SYCO23 ecosystem as two primary UI projects inside one shared brand system:

1. syco23-webplayer — public listening surface.
2. syco23-transmission — operator multistream control surface.

The supporting non-UI projects are azuracast on Linode VPS and syco23bot + syco23adminbot on Telegram. These supporting projects must still inherit the same tone, hierarchy, and information design language in dashboards, messages, alerts, and generated assets.

Shared visual language

- Mechanical totem.
- Ancient visual energy.
- Large sound-system scale.
- Dark mineral surfaces.
- One dominant key color per screen or mode.
- Bold condensed industrial display type.
- Clean readable body text.
- Sparse signal-like motion only.

Avoid terminal aesthetics, neon rave art, cyberpunk, glossy startup visuals, soft bubbly cards, generic dashboard layouts, and decorative glitch.

syco23-webplayer layout guidance

- Mobile portrait first.
 - Full-screen immersive listening surface with stacked content hierarchy.
 - Clear now-playing and live-state visibility.
 - Strong fallback states for offline, reconnecting, and maintenance.
 - Background art and overlays must never break readability.
 - Future-ready slot for optional visual preview player.
-

syco23-transmission layout guidance

- Desktop control surface must be scroll-free or nearly scroll-free in the primary live mode.
 - Layout centered around a control spine with source state, master transmission state, and destination matrix.
 - Providers appear as transmission nodes, not social media promo tiles.
 - Secondary details live in drawers or adjacent panels.
 - Template gallery must show meaningful generated preview images.
 - Colorized log view must feel like telemetry, not chat or notification feed.
 - Built stream preview player must look like a monitor window, not a generic browser video box.
-

Telegram bot message guidance

- Bot replies must be direct, compact, and operational.
- Public bot messages should be clear and easy to scan.

- Admin bot alerts should prioritize state, error, and actionability.
 - Messaging should reflect SYCO23 voice without sounding corporate or playful.
-

Cross-project consistency

Design all surfaces so they feel like parts of the same machine: the public player, the transmission app, the backend/admin utilities, and the bots should all read as one ecosystem.

SYCO23 V4.1 — AGENT BUILD PROMPT

Build the SYCO23 system as four connected projects, not as one monolith:

1. syco23-webplayer
 2. syco23-transmission
 3. azuracast (installed on Linode VPS)
 4. syco23bot + syco23adminbot
-

Shared rules

- Follow SYCO23 v4.1 brand language: mechanical totem, ancient energy, sound-system scale, dark mineral surfaces, restrained signal motion, industrial typography.
 - No terminal, neon rave, cyberpunk, glossy startup, or generic SaaS dashboard aesthetics.
 - Keep project boundaries explicit.
 - Use shared contracts between projects.
-

Build scope by project

syco23-webplayer

Build a public listening surface with live playback, metadata, schedule, archive, offline/reconnecting states, and future-ready public preview integration.

syco23-transmission

Build the operator multistream app with:

- template gallery and generated previews
- custom template builder with user backgrounds
- custom output profiles
- provider routing for YouTube, Telegram, TikTok, Twitch, Instagram, Mixer, Mixcloud, Facebook, and custom RTMP
- provider transmission kits
- frontend video preview player
- colorized logs
- SQLite persistence
- FFmpeg pipeline manager
- ultra-efficient watchdog
- AzuraCast metadata middleware

azuracast

Treat AzuraCast on Linode as the radio core and public API source. Do not expose it directly to all frontends. Use middleware and normalized contracts.

syco23bot + syco23adminbot

Build Telegram bots for public now-playing/link queries and operator alerts/status queries/safe remote actions.

Technical rules

- Nuxt 3 + Vue 3 + TypeScript for the web projects.
 - SQLite for persistence where the transmission app needs local operational data.
 - FFmpeg as child-process pipeline manager.
 - Long-running services must not be assigned to Vercel serverless.
 - AzuraCast metadata must be consumed through server-side middleware.
 - Bots must consume normalized internal APIs.
-

Output expectation

Produce architecture, code, and documentation that clearly separate the four projects while keeping them interoperable.

SYCO23 V4.1 — SHORT PRODUCTION PROMPT

Build SYCO23 as four connected projects:

- syco23-webplayer — public live player, metadata, schedule/archive, fallback states, future-ready public preview support
- syco23-transmission — multistream control app with template previews, custom templates, provider output profiles, transmission kits, preview player, colorized logs, SQLite, FFmpeg, watchdog, metadata middleware
- azuracast — station backend on Linode VPS and baseline public API source
- syco23bot + syco23adminbot — Telegram public bot + admin bot for alerts and remote status

Shared rules: SYCO23 v4.1 brand only, mobile-first, no terminal/neon/cyberpunk/glossy SaaS styling, explicit project boundaries, normalized APIs between services, no long-running FFmpeg/watchdog on Vercel serverless.

SYCO23 V4.1 — LONG REFERENCE PACK

1. Version change

Version 4.1 updates the previous v4 package by splitting the ecosystem into four projects and by adding a dedicated user-stories backlog document.

2. Project split

syco23-webplayer

Public listening UI.

Main features

- Live audio playback
- metadata display
- archive and schedule views
- reconnecting/offline/fallback states
- future-ready visual preview path
- shared SYCO23 brand system

syco23-transmission

Operator-facing multistream webapp.

Main features

- template gallery with generated previews
- custom template builder with user backgrounds
- custom provider output profiles
- routing to YouTube, Telegram, TikTok, Twitch, Instagram, Mixer, Mixcloud, Facebook, custom RTMP
- provider transmission kits
- built stream preview player
- colored logs
- SQLite persistence
- FFmpeg pipelines
- watchdog supervision
- AzuraCast metadata middleware

azuracast

Linode VPS radio core.

Main features

- station backend
- source stream foundation
- public API baseline
- stable service and recoverability

syco23bot + syco23adminbot

Telegram bot layer.

Main features

- public now-playing access
 - public link distribution
 - admin alerts
 - admin status queries
 - safe remote actions
 - bot audit trail
-

3. Shared architecture principles

- Mechanical totem brand system across all surfaces.
 - Mobile-first where UI exists.
 - Shared normalized contracts between projects.
 - Middleware in front of AzuraCast when richer metadata/stats are required.
 - Long-running pipeline and watchdog services must run on infrastructure that supports persistent processes.
-

4. Document map

- Master spec — system boundaries and main requirements.
 - Design layout prompt — UI language for webplayer and transmission.
 - Agent build prompt — implementation starter.
 - Short production prompt — compact agent directive.
 - User stories — separate implementation backlog by project.
-

5. Primary implementation order

1. AzuraCast baseline and integration contract.
 2. Transmission middleware, persistence, FFmpeg, watchdog.
 3. Webplayer public surface.
 4. Telegram bots.
 5. Cross-project polish and shared contracts.
-

SYCO23 V4.1 — USER STORIES

This document extracts the main feature requirements from the evaluated SYCO23 v4.1 guideline set and rewrites them as project-separated user stories for implementation planning.

Project map

The SYCO23 v4.1 ecosystem is split into four connected projects:

1. syco23-webplayer — public listening surface and future public stream preview surface.
2. syco23-transmission — operator-facing multistream control webapp.
3. azuracast — radio core installed on the Linode VPS, used as station backend, ingest, public API source, and stream source.
4. syco23bot + syco23adminbot — Telegram bot layer for public listeners and internal operators.

All projects must follow the same v4.1 brand language: mechanical totem, ancient energy, sound-system scale, dark mineral surfaces, one key color per surface, industrial typography, and anti-commercial restraint.

syco23-webplayer

Epic: Public listening experience

US-WP-01 — Live playback

As a listener, I want a reliable live player with fast startup and stable playback so I can listen to SYCO23 on mobile and desktop without friction.

Acceptance criteria

- Player starts quickly from the primary stream source.
- Playback state survives navigation where technically possible.
- Clear online, reconnecting, offline, and standby states are shown.
- Fallback messaging exists when the stream is unavailable.

US-WP-02 — Mobile-first layout

As a listener, I want the player to feel native on portrait mobile first so I can use it comfortably during travel, at parties, or on weak devices.

Acceptance criteria

- Portrait layout is the primary design target.
- Desktop, tablet, and TV-safe variants inherit the same brand logic.
- Controls remain large, readable, and thumb-safe.
- Reduced-motion mode is supported.

US-WP-03 — Metadata visibility

As a listener, I want now-playing metadata, artwork, and stream state to update clearly so I always know what is live.

Acceptance criteria

- Title, artist, show, artwork, and live state update from shared metadata.

- Metadata failures do not break playback.
- Long text remains readable without destroying layout hierarchy.

US-WP-04 — Brand-consistent listening surface

As a brand owner, I want the webplayer to look like SYCO23 v4.1 instead of a generic audio app so the listening surface strengthens identity.

Acceptance criteria

- Mechanical-totem layout language is visible in framing and hierarchy.
- No terminal, neon rave, cyberpunk, or glossy SaaS aesthetics appear.
- One dominant key color is used per page or mode.

US-WP-05 — Public video preview readiness

As an operator, I want the public webplayer architecture to be ready for future embedded video preview support so the same system can later expose built streams visually.

Acceptance criteria

- Component boundaries allow future embedding of a preview player.
- HLS-ready rendering path is considered in the architecture.
- UI remains usable even if video preview is disabled in v1.

US-WP-06 — Status and fallback states

As a listener, I want clear fallback and maintenance states so I understand whether the system is offline, reconnecting, or in maintenance.

Acceptance criteria

- Dedicated offline, reconnecting, and maintenance surfaces exist.
- Status communication is not color-only.
- Recovery path or next action is visible where possible.

Epic: Public information

US-WP-07 — Schedule and archive access

As a listener, I want to access schedule and archive views so I can see what is coming and what already aired.

Acceptance criteria

- Schedule view supports upcoming shows.
- Archive view supports prior sessions or mixes.
- Both views reuse the same design tokens and shell logic.

syco23-transmission

Epic: Multistream operations

US-TX-01 — Create a stream from templates

As an operator, I want to create a stream by choosing from templates with generated previews so I can launch faster and compare visual approaches before going live.

Acceptance criteria

- Template gallery appears during stream creation.
- Each template shows a generated preview image.

- Dummy text is acceptable in early previews, but layout and brand treatment must be real.
- Template choice is stored with the stream configuration.

US-TX-02 — Build custom templates

As an operator, I want to build my own template with my own background images so I can tailor transmission visuals to different sessions while keeping the SYCO23 system intact.

Acceptance criteria

- User can upload and manage custom background assets.
- Custom templates remain responsive across viewport modes.
- Provider-aware layout adaptation is supported.
- Templates are persisted in SQLite.

US-TX-03 — Custom output profiles

As an operator, I want reusable provider-specific output profiles so I can launch the correct format for YouTube, Telegram, and future targets without rebuilding settings each time.

Acceptance criteria

- Built-in presets include at least 16:9 YouTube and Telegram-adapted outputs.
- Users can create, edit, clone, and reuse profiles.
- Profiles connect directly to FFmpeg pipeline generation.

US-TX-04 — Multistream distribution

As an operator, I want to route one program feed to multiple providers at once so I can run one-to-many transmissions from a single control surface.

Acceptance criteria

- Supported providers include YouTube, Telegram, TikTok, Twitch, Instagram, Mixer, Mixcloud, and Facebook.
- Future custom RTMP targets are supported.
- Provider state model includes idle, configured, armed, connecting, live, degraded, failed, cooldown, and disabled.

US-TX-05 — Provider transmission kits

As an operator, I want an individual transmission kit for each configured provider so I can copy and paste the right titles, descriptions, labels, and launch notes instantly.

Acceptance criteria

- Transmission kits are generated from provider + template + brand context.
- Kit content is copy-paste ready.
- Kits are stored persistently and regenerable.
- Tone and formatting stay inside SYCO23 v4.1 rules.

US-TX-06 — Built stream preview

As an operator, I want a frontend video player to preview the built live stream so I can verify output before and during distribution.

Acceptance criteria

- HLS playback is supported at minimum.
- Player is embedded in the operator UI.
- Architecture allows future use in the public webplayer.
- Player follows SYCO23 styling rather than browser-default chrome.

Epic: Control, monitoring, and reliability

US-TX-07 — Colorized logs

As an operator, I want a colorized real-time log so I can spot warnings, failures, and recovery events instantly during a live session.

Acceptance criteria

- Log levels include info, warning, error, success, and debug.
- Logs are filterable by source and level.
- Logs persist in SQLite.
- Logs are usable on mobile and desktop.

US-TX-08 — Watchdog supervision

As an operator, I want an ultra-efficient watchdog service so stalled pipelines or failed provider sessions are detected and recovered without constant manual intervention.

Acceptance criteria

- Watchdog monitors ingest, FFmpeg, and provider sessions.
- Threshold-based detection is configurable.
- Recovery actions are logged and surfaced to the UI.
- Watchdog overhead remains low.

US-TX-09 — Persistent state

As an operator, I want all critical entities stored persistently so the system can survive restarts without losing operational context.

Acceptance criteria

- Streams, providers, templates, output profiles, kits, schedules, logs, assets, and events are stored in SQLite.
- System can restore useful state after restart.
- Historical session data remains queryable.

US-TX-10 — Mobile-first operator surface

As an operator, I want the control app to remain mobile-first and fully responsive so I can operate or monitor sessions from phone, tablet, laptop, or TV-safe display.

Acceptance criteria

- Portrait mobile is a first-class operational layout.
- Desktop remains scroll-free or nearly scroll-free in the primary live-control surface.
- Tablet and TV-safe modes are supported.

US-TX-11 — Metadata middleware

As an operator, I want the transmission app to use its own metadata middleware so the frontend can consume normalized and enriched AzuraCast data instead of relying only on the public API surface.

Acceptance criteria

- Frontend never calls AzuraCast directly.
- Middleware polls AzuraCast server-side.
- Middleware enriches and stores metadata snapshots in SQLite.
- Custom stats and queries are exposed through internal API routes.

azuracast (Linode VPS)

Epic: Radio core and source-of-truth services

US-AZ-01 — Stable station backend

As an infrastructure operator, I want AzuraCast running on the Linode VPS as the central radio backend so SYCO23 has a stable source for stream ingest, station control, and baseline metadata.

Acceptance criteria

- AzuraCast is installed and maintained on Linode.
- Core station services survive restart.
- Access is restricted appropriately.

US-AZ-02 — Public API source

As a system integrator, I want AzuraCast to provide baseline public API data so the rest of the SYCO23 stack can build on a known source of truth.

Acceptance criteria

- Public API endpoints are available to the middleware layer.
- Metadata and station state remain consumable through normalized middleware contracts.
- AzuraCast is not exposed directly to every frontend client.

US-AZ-03 — Enriched stats workflow

As an operator, I want custom statistics beyond the public AzuraCast API so I can analyze performance, stream health, and history in more detail.

Acceptance criteria

- Middleware can enrich AzuraCast-derived data into SQLite.
- Custom queries support additional stats not exposed publicly.
- This enrichment does not require risky direct frontend DB access.

US-AZ-04 — Reliable stream source

As the transmission system, I want AzuraCast to behave as a dependable source input so outbound multistream routing starts from a stable base.

Acceptance criteria

- Ingest or relay source is clearly defined.
- Failure states can be detected by downstream watchdog systems.
- Source URLs and credentials are managed securely.

US-AZ-05 — Backup and recovery

As an infrastructure operator, I want backup and recovery procedures for AzuraCast and related state so a VPS incident does not wipe the station core.

Acceptance criteria

- Backup strategy exists for configs and relevant persistent data.
- Recovery steps are documented.
- Service health can be monitored externally.

syco23bot + syco23adminbot

Epic: Public Telegram access

US-BOT-01 — Listener-facing now playing bot

As a Telegram user, I want a public SYCO23 bot that can tell me what is live now so I can check the stream without opening the full site.

Acceptance criteria

- Bot can return live state, current track/show, and listening link.
- Messaging tone follows SYCO23 brand voice without sounding corporate.
- Public bot actions stay simple and clear.

US-BOT-02 — Public stream and link access

As a Telegram user, I want the bot to give me links to the webplayer, stream, schedule, and other public surfaces so I can jump directly into the ecosystem.

Acceptance criteria

- Bot can return current stream link and public webplayer link.
- Bot can expose key public destinations with minimal command friction.

Epic: Operator Telegram control

US-BOT-03 — Admin alerts

As an operator, I want syco23adminbot to send alert messages when the watchdog detects failures or important state changes so I do not need to watch the dashboard constantly.

Acceptance criteria

- Alerts include provider failures, recovery actions, source outages, and critical watchdog events.
- Alert formatting is concise and readable on mobile.
- Alert routing supports role-based recipients or approved chats.

US-BOT-04 — Quick admin status queries

As an operator, I want to ask the admin bot for current transmission status so I can check health from Telegram when away from the main interface.

Acceptance criteria

- Admin bot can report source status, active destinations, major errors, and watchdog status.
- Responses pull from normalized internal APIs rather than scraping frontend UI.

US-BOT-05 — Controlled remote actions

As an authorized operator, I want approved remote actions through the admin bot so I can trigger safe operational commands without opening the full app.

Acceptance criteria

- Actions are permission-gated.
- Actions are logged.
- Only safe commands are exposed in early versions.
- Destructive actions require confirmation.

US-BOT-06 — Bot audit trail

As a system owner, I want bot interactions to be logged so operator actions and alerts can be reviewed later.

Acceptance criteria

- Important inbound commands and outbound alerts are logged.
- Logs can be correlated with stream sessions and watchdog events.

Cross-project stories

US-X-01 — Shared brand system

As a system owner, I want all projects to use the same SYCO23 v4.1 brand rules so the ecosystem feels like one coherent machine rather than separate tools.

US-X-02 — Shared data contracts

As a developer, I want clean API and event contracts between AzuraCast, transmission, webplayer, and bots so the ecosystem can evolve without breaking every project.

US-X-03 — Shared deployment clarity

As an operator, I want deployment boundaries documented per project so I know what runs on Linode, what runs in Docker, what can run on Vercel, and what must stay as long-running services.

SYCO23 V4.1 — ROADMAP

This roadmap groups the SYCO23 v4.1 ecosystem into milestone-based delivery stages across the four connected projects:

1. syco23-webplayer
2. syco23-transmission
3. azuracast
4. syco23bot + syco23adminbot

The roadmap assumes this dependency order:

1. AzuraCast baseline and integration contract
 2. Transmission middleware, persistence, FFmpeg, and watchdog
 3. Webplayer public surface
 4. Telegram bot layer
 5. Cross-project refinement and production hardening
-

Milestone structure

MVP

Goal: get the radio core stable, the transmission path usable, and the public player online.

Beta

Goal: make the system operator-ready with custom workflows, stronger recovery, and more complete cross-project integration.

Production

Goal: harden the full ecosystem for dependable long-running use, cleaner automation, and polished public/admin surfaces.

MVP

azuracast

Target outcome

- AzuraCast installed and stable on the Linode VPS
- Stream source and public API baseline available
- Access, restart, and backup procedure documented

Deliverables

- Running AzuraCast instance on Linode
- Confirmed public Now Playing/API access path
- Environment and credential inventory

- Basic backup and restart checklist

Success criteria

- Station survives restart cleanly
- Public API is reachable by the transmission middleware
- Source stream is usable as the baseline ingest/input layer

syco23-transmission

Target outcome

- Operator can configure providers, launch a stream, and monitor basic health
- System persists essential state in SQLite
- FFmpeg and watchdog foundation exists

Deliverables

- Root app shell and token system
- Provider matrix for YouTube, Telegram, TikTok, Twitch, Instagram, Mixer, Mixcloud, Facebook
- Basic stream creation flow
- Initial template gallery with generated previews
- Initial output profile system with built-in presets
- SQLite persistence for streams, providers, profiles, templates, and logs
- AzuraCast metadata middleware (/api/metadata, /api/metadata/stats)
- FFmpeg pipeline manager foundation
- Watchdog foundation with critical event logging
- Colorized log view (basic)

Success criteria

- Operator can arm and configure providers
- Operator can launch a transmission from a saved setup
- Metadata reaches the UI through middleware, not direct frontend API calls
- Critical state persists across restart
- Failures are visible in logs and status surfaces

syco23-webplayer

Target outcome

- Public listener can open the player and listen reliably
- Live metadata and fallback states are visible

Deliverables

- Public live player
- Mobile-first responsive layout
- Metadata stack (title, artist, show, artwork, live state)
- Offline/reconnecting/maintenance states
- Schedule and archive placeholders or initial surfaces

Success criteria

- Listener can access live playback on mobile and desktop
- Metadata updates do not break playback
- Fallback states are readable and branded

syco23bot + syco23adminbot

Target outcome

- Public bot can answer now-playing questions
- Admin bot can deliver critical alerts

Deliverables

- Public bot: now playing + stream link
- Admin bot: critical watchdog and provider failure alerts
- Basic command/auth structure
- Bot event logging foundation

Success criteria

- Public bot returns useful live status
 - Admin bot sends actionable critical alerts
 - Bot messages remain compact and readable on mobile
-

Beta

azuracast

Target outcome

- AzuraCast is operationally reliable and fully integrated into the middleware strategy

Deliverables

- Verified API contract list for middleware polling
- Improved backup cadence and restore test
- Monitoring hooks for service health

Success criteria

- Middleware has stable upstream contracts
- Recovery steps are validated, not theoretical

syco23-transmission

Target outcome

- Operators can run richer branded workflows and trust the system more during live use

Deliverables

- Custom template builder with user-uploaded backgrounds
- Reusable custom output profiles
- Provider-specific transmission kit generator
- Built-in frontend video preview player (SycoVideoPlayer.vue)
- Expanded watchdog recovery actions
- Filterable colored logs by source and level
- Archive and schedule views with usable data
- Mobile-first operator mode polished
- Tablet and TV-safe layouts introduced

Success criteria

- Operator can create streams from templates or custom templates
- Operator can generate and copy provider-specific transmission kits
- Operator can preview the built stream visually

- Recovery behavior reduces manual intervention in common failures

syco23-webplayer

Target outcome

- Public player becomes a coherent branded destination, not only a functional stream surface

Deliverables

- Finalized schedule and archive views
- Stronger background/artwork handling
- Shared API contracts aligned with transmission middleware
- Future-ready slot for public visual preview component

Success criteria

- Public player feels consistent with the transmission ecosystem
- Schedule/archive are useful, not placeholder-only

syco23bot + syco23adminbot

Target outcome

- Bots become part of daily operation instead of emergency-only tools

Deliverables

- Admin status queries
- Public link distribution improvements
- Permission-gated safe remote actions
- Better audit trail for commands and alerts

Success criteria

- Operator can query current transmission state from Telegram
- Public bot improves access to the player and schedule
- Audit trail connects bot events to stream/watchdog history

Production

azuracast

Target outcome

- Radio core is hardened and operationally boring in the best way

Deliverables

- Stable maintenance workflow
- Documented incident response path
- Refined health monitoring and backup verification

Success criteria

- AzuraCast behaves as a dependable long-running upstream service
- Restore and failover knowledge is documented and reusable

syco23-transmission

Target outcome

- Transmission app is dependable for regular live use and scaled operational complexity

Deliverables

- Full provider workflow hardening
- Production-grade watchdog thresholds and recovery matrix
- Better observability for FFmpeg, provider state, and metadata health
- Refined role/permission boundaries where needed
- Final responsive polish across phone, desktop, tablet, and TV-safe display
- Documentation for deployment, recovery, and extension

Success criteria

- Operators can trust the system during repeated live sessions
- Most common failures are surfaced early and handled predictably
- Logs, kits, templates, profiles, and session history provide real operational value

syco23-webplayer

Target outcome

- Public player is fully brand-consistent, stable, and extensible

Deliverables

- Public visual preview integration path clarified or implemented
- Final design polish and accessibility pass
- Stable archive/schedule/public metadata flow

Success criteria

- Public experience feels finished and dependable
- Webplayer can evolve without breaking shared contracts

syco23bot + syco23adminbot

Target outcome

- Bot layer is trustworthy for both public convenience and operator workflows

Deliverables

- Hardened permissions and logging
- Cleaner operator commands
- Better alert routing and suppression rules

Success criteria

- Bots reduce operator friction instead of adding noise
- Alerting stays actionable and not spammy

Cross-project milestones

MVP cross-project goals

- Shared naming and brand rules applied everywhere
- Initial normalized API contracts defined
- Basic deployment boundaries documented

Beta cross-project goals

- Shared data contracts stabilized
- Public/player/admin surfaces feel like one ecosystem
- Core documentation exists per project

Production cross-project goals

- Clear recovery playbooks exist
 - Project ownership boundaries are documented
 - Deployment, monitoring, and extension paths are repeatable
-

Recommended build order

1. azuracast baseline on Linode
 2. syco23-transmission MVP core: middleware, SQLite, provider matrix, FFmpeg foundation, watchdog foundation
 3. syco23-webplayer MVP public listening surface
 4. syco23bot + syco23adminbot MVP alerts and now-playing
 5. syco23-transmission Beta branded operator workflows
 6. syco23-webplayer Beta archive/schedule polish
 7. bots Beta admin query and safe actions
 8. Production hardening across all projects
-

Exit criteria by milestone

MVP exit criteria

- AzuraCast stable on Linode
- Transmission app can configure and launch core provider flows
- Webplayer can play live stream and show metadata
- Bots can deliver minimum useful public/admin value

Beta exit criteria

- Operators can use templates, profiles, kits, preview player, and richer monitoring
- Public player has useful schedule/archive and stronger identity execution
- Admin bot supports meaningful operational queries

Production exit criteria

- All projects are documented, monitored, recoverable, and dependable in routine use
- Cross-project contracts are stable enough for ongoing iteration without chaos